

---

# STRUCTOPTIMA

---

## Core Logic & Code Architecture Reference Manual

|                   |                                          |
|-------------------|------------------------------------------|
| Document Type:    | Code Architecture & Algorithm Reference  |
| Total Modules:    | 15 source modules documented             |
| Applicable Codes: | IS 456, IS 1893, IS 875, IS 13920, SP 34 |
| Prepared By:      | StructOptima Development Team            |
| Version:          | v1.0.0 (September 2026)                  |

---

## TABLE OF CONTENTS

1. `cad_parser.py` - Parses DXF files to extract structural geometry (columns, be...
2. `cad_loader.py` - Adapter between the DXF parser and the structural data model...
3. `auto_framer.py` - Generates complete structural framing (columns + beams) from...
4. `column_placer.py` - Professional IS-code column placement engine. Enforces manda...
5. `beam_placer.py` - 5-phase beam placement and sizing engine. Phase 1: column-to...
6. `grid_manager.py` - Central structural engine. Manages columns, walls, slabs, fo...
7. `member_design.py` - Implements IS 456:2000 limit state design for beams, columns...
8. `rebar_detailer.py` - Converts required steel areas into practical bar bending sch...
9. `foundation_eng.py` - Designs isolated square RC spread footings. Sizes plan area ...
10. `seismic.py` - IS 1893:2016 equivalent static seismic analysis. Calculates ...
11. `wind_load.py` - IS 875:2015 Part 3 wind load analysis. Calculates design win...
12. `load_combinations.py` - Generates IS 456:2000 Table 18 limit state load combinations...
13. `quantifier.py` - Generates Bill of Materials (BOM), cost estimation, carbon f...
14. `stability.py` - Structural stability and safety checks. Column and beam slen...
15. `materials.py` - Defines structural material models as Pydantic BaseModel cla...

# 1. CAD\_PARSER.PY

**File:** src/cad\_parser.py

Parses DXF files to extract structural geometry (columns, beams, walls). Uses a robust 3-tier fallback strategy: Tier 1 checks named wall layers, Tier 2 scans all layers, Tier 3 performs raw geometric entity scan. Normalizes double-line architectural walls into single structural centerlines.

## Key Classes & Data Models

- CADParser: filepath, doc, msp, layer\_map

## Key Functions

- load() -> None: Opens and parses the DXF file using ezdxf
- extract\_walls() -> List[Tuple]: 3-tier wall extraction from DXF layers
- normalize\_to\_centerlines(walls, thickness\_range, angle\_tol) -> List: Merges parallel wall pairs into structural centerlines
- detect\_wall\_thickness(walls) -> float: Auto-detects wall thickness from parallel line spacing
- detect\_enclosed\_rectangles() -> List: Finds elevator shafts and stairwells

## Core Algorithm (Step-by-Step)

extract\_walls() Algorithm:

1. Check for layers with wall keywords (WALL, STRUCT, MASONRY)
2. Tier 1: Extract LINE/POLYLINE from keyword layers
3. Tier 2: If empty, scan ALL layers for line entities
4. Tier 3: If empty, inspect all modelspace entities
5. Auto-detect units (mm if max\_coord > 300, else meters)
6. Filter segments shorter than min\_length (100mm)

normalize\_to\_centerlines() Algorithm:

1. For each wall pair (i, j), check if parallel (angle < 8 deg)
2. Check if perpendicular distance is within thickness\_range
3. If matched: average endpoints to get centerline
4. If unmatched: treat as single-line wall

**IS Code References:** Architectural drafting standards (115mm, 230mm, 350mm wall thicknesses)

---

## 2. CAD\_LOADER.PY

**File:** src/cad\_loader.py

Adapter between the DXF parser and the structural data models. Auto-detects drawing units (mm vs m), snaps entities to structural grid lines, delegates to AutoFramer and ColumnPlacer for automated column/beam placement, and builds the GridManager with walls.

### Key Classes & Data Models

- CADLoader: parser, placement\_warnings, scale\_factor, framer, placer

### Key Functions

- load\_grid\_manager(auto\_frame, seismic\_zone) -> (GridManager, List[StructuralMember])
- get\_placement\_json() -> str: Returns JSON summary of placement results

### Core Algorithm (Step-by-Step)

```
load_grid_manager() Algorithm:  
1. If auto_frame: run AutoFramer -> ColumnPlacer -> generate beams  
2. Auto-detect scale: if max_coord > 500, scale = 0.001 (mm->m)  
3. Scale all column coordinates to meters  
4. Extract unique X,Y coords to form grid lines  
5. Snap each column to nearest grid line (0.1m tolerance)  
6. Extract walls, detect thickness (default 230mm)  
7. Create Wall objects with opening_fraction = 0.33  
8. Return populated (GridManager, List[StructuralMember])
```

**IS Code References:** IS 13920:2016 (seismic column sizing), IS 456:2000 (minimum member sizes)

### 3. AUTO\_FRAMER.PY

**File:** src/auto\_framer.py

Generates complete structural framing (columns + beams) from architectural wall centerlines. Identifies structural nodes at wall intersections, inserts intermediate nodes for spans > 5.0m, filters non-structural spurs, snaps to orthogonal grid, and establishes beam connectivity.

#### Key Classes & Data Models

- AutoFramer: parser, nodes, columns, beams, centerlines, max\_span\_mm=5000

#### Key Functions

- identify\_structural\_nodes() -> None: Finds column locations from wall intersections
- generate\_beams(use\_fallback, use\_is\_code, seismic\_zone) -> List[StructuralMember]

#### Core Algorithm (Step-by-Step)

```
identify_structural_nodes() Algorithm:
1. Extract normalized wall centerlines
2. Collect wall endpoints as candidate nodes
3. For walls > 5000mm: insert intermediate nodes at L/n spacing
4. Cluster nearby candidates within 1000mm -> use centroid
5. Filter spurs (degree-1 nodes with short connection < 1200mm)
6. Filter collinear joints (degree-2, angle ~180 deg)
7. Snap to orthogonal grid, merge nodes < 600mm apart

generate_beams() Algorithm:
1. Walk each X grid line: connect adjacent columns with beams
2. Walk each Y grid line: connect adjacent columns with beams
3. Verify wall connectivity for each beam pair
4. Ensure minimum 2 beams per column (add fallback beams if needed)
5. Size: 250x450mm for span<=5m, 250x600mm for span>5m
```

**IS Code References:** IS 456:2000 Table 26 (economical spans <= 5.0m)

## 4. COLUMN\_PLACER.PY

**File:** src/column\_placer.py

Professional IS-code column placement engine. Enforces mandatory corner columns, junction-priority ordering (Cross > T > Corner > Edge > Interior), grid coverage analysis, gap filling for spans > max\_span, formwork-standard dimension snapping, and floating column detection.

### Key Classes & Data Models

- JunctionType(Enum): CORNER, EDGE, T\_JUNCTION, CROSS\_JUNCTION, INTERIOR
- ColumnPlacement: id, x, y, width, depth, junction\_type, is\_floating, reinforcement\_rule
- ColumnPlacer: SEISMIC\_MIN\_DIM=300, NON\_SEISMIC\_MIN=230, MIN\_ASPECT\_RATIO=0.4

### Key Functions

- generate\_placement() -> PlacementResult: Full column placement pipeline
- detect\_floating\_columns(lower\_floor\_cols) -> List[Warning]

### Core Algorithm (Step-by-Step)

```
generate_placement() Algorithm:  
1. Calculate node degrees from incident wall centerlines  
2. Classify junctions: degree>=4->Cross, 3->T, 2+perimeter->Corner  
3. Extract primary grid lines (coverage >= 55%)  
4. Place mandatory corner columns (special confining steel)  
5. Place at Cross junctions, then T-junctions  
6. Place at primary grid intersections where walls exist  
7. Fill gaps > max_span (5000mm) along perimeter/walls  
8. Deduplicate within proximity distance  
9. Size columns: min 300mm (seismic), snap to formwork sizes  
10. Orient: depth parallel to larger span direction  
11. Return PlacementResult with warnings
```

**IS Code References:** IS 456:2000 Cl 26.5.3; IS 13920:2016 Cl 6.1.3 (300mm min), Cl 7.1.2 (b/D >= 0.4), Cl 7.4 (confining steel)

## 5. BEAM\_PLACER.PY

**File:** src/beam\_placer.py

5-phase beam placement and sizing engine. Phase 1: column-to-column primary beams. Phase 2: long span evaluation (> 7m). Phase 3: slab panel detection with secondary beam bisection. Phase 4: cantilever detection with back-span safety. Phase 5: depth sizing via IS 456 L/d ratios.

### Key Classes & Data Models

- BeamType(Enum): PRIMARY, SECONDARY, CANTILEVER, WALL\_SUPPORT
- BeamPlacement: id, beam\_type, span\_mm, width\_mm, depth\_mm, support\_type
- BeamPlacer: MAX\_SLAB\_SPAN=6000, MAX\_CANTILEVER=2500, MIN\_BD\_RATIO=0.3

### Key Functions

- generate\_placement() -> PlacementResult: Full beam placement pipeline
- \_calculate\_depth(span\_mm, support\_type) -> float: IS 456 L/d sizing
- \_detect\_cantilevers() -> None: Find and validate cantilever beams

### Core Algorithm (Step-by-Step)

`_calculate_depth()` Algorithm:

1. Cantilever:  $\text{depth} = \text{span} / 7$
  2. Secondary (beam-beam):  $\text{depth} = \text{span} / 10$
  3. Primary (col-col):  $\text{depth} = \text{span} / 12$
  4. Minimum depth = 400mm
  5. Round UP to nearest 50mm
- Note: span converted to mm before sizing (bug fix applied)

`_detect_cantilevers()` Algorithm:

1. Find perimeter columns with slab projecting 500-2500mm beyond
2. Compute back-span from opposite framing beam
3. Verify  $\text{back\_span} \geq 1.5 * \text{cantilever\_span}$
4. If unsafe: CRITICAL stability warning
5. Assign DOUBLY\_REINFORCED logic

**IS Code References:** IS 456:2000 Cl 23.2 Table 23 (L/d ratios), Cl 29 (deep beams); IS 13920:2016 Cl 6.1.2 ( $b/D \geq 0.3$ )

## 6. GRID\_MANAGER.PY

**File:** src/grid\_manager.py

Central structural engine. Manages columns, walls, slabs, footings, and multi-story systems. Handles bay count optimization (3-5m rule), tributary area calculation with void deductions, cumulative top-to-bottom load takedown, column size optimization per IS 456 Cl 39.3, and delegates rebar detailing.

### Key Classes & Data Models

- Column(BaseModel): id, x, y, level, trib\_area\_m2, load\_kn, width\_nb, depth\_nb
- Wall(BaseModel): id, start/end coords, thickness\_mm, height\_m, opening\_fraction
- GridManager(BaseModel): width\_m, length\_m, num\_stories, story\_height\_m, columns, walls, beam/slab/rebar\_schedule

### Key Functions

- calculate\_trib\_areas(staircase\_bay, void\_zones) -> None
- calculate\_loads(floor\_load\_kn\_m2, wall\_load\_kn\_m) -> None
- optimize\_column\_sizes(concrete, fy, seismic\_zone) -> None
- detail\_beams(beams) -> None: Delegates to RebarDetailer
- detail\_columns(concrete) -> None: Delegates to RebarDetailer
- detail\_slabs() -> None: Delegates to RebarDetailer

### Core Algorithm (Step-by-Step)

```
calculate_trib_areas() Algorithm:
1. Inject ALL column positions into grid lines (Bug 1 fix)
2. For each column, find its grid indices in sorted lists
3. width_X = (x[i]-x[i-1])/2 + (x[i+1]-x[i])/2
4. width_Y = (y[j]-y[j-1])/2 + (y[j+1]-y[j])/2
5. trib_area = width_X * width_Y
6. Deduct void zone contributions (25% per void corner)

calculate_loads() Algorithm:
1. Pre-distribute wall loads: 50% to each end column
2. Group columns by (x,y) into vertical stacks
3. For each stack, top to bottom:
  a. Floor load = trib_area * floor_load_kn_m2
  b. Wall load = beam_len * wall_load_kn_m
  c. Self-weight = w * d * h * 25 kN/m3
  d. Accumulate total -> col.load_kn

optimize_column_sizes() Algorithm:
1. Pu = col.load_kn * 1.5 * 1000 (factored, in N)
2. Solve: Pu_cap = 0.4*fck*Ac + 0.67*fy*Asc >= Pu
3. Start at 230x230mm, increment by 50mm until satisfied
4. Enforce seismic min (300mm for Zone III/IV/V)
5. Snap to formwork sizes [230,300,350,400,450,500,600]
6. Stack locking: all levels use max size in the stack
```

**IS Code References:** IS 456:2000 Cl 26.5.3, Cl 39.3, Table 18, Table 26; IS 13920:2016 Cl 6.1.3; IS 875 (Part 1)

## 7. MEMBER\_DESIGN.PY

**File:** src/member\_design.py

Implements IS 456:2000 limit state design for beams, columns, and slabs. Beam flexure (singly/doubly reinforced), shear with exact Table 19  $\tau_c$  lookup, column biaxial P-M interaction (Bresler CI 39.6), slenderness checks, and minimum eccentricity.

### Key Classes & Data Models

- MemberDesigner: concrete\_grade, steel\_grade, clear\_cover\_mm
- BeamDesignResult: flexure results, shear results, detailing
- ColumnDesignResult: P-M interaction, slenderness, ties

### Key Functions

- design\_beam\_flexure(b, D, Mu) -> (ReinfType, Ast, Asc)
- design\_beam\_shear(b, D, Vu, Ast) -> ( $\tau_v$ ,  $\tau_c$ , sv, Asv)
- check\_pm\_interaction(Pu, Mux, Muy, b, D, Ast) -> ratio
- design\_column(b, D, L, Pu, Mux, Muy) -> ColumnDesignResult

### Core Algorithm (Step-by-Step)

design\_beam\_flexure() Algorithm:

1.  $d = D - \text{cover} - \text{bar\_dia}/2$
2.  $\mu_{lim} = 0.36 \cdot f_{ck} \cdot b \cdot d^2 \cdot (x_{u\_max}/d) \cdot (1 - 0.42 \cdot x_{u\_max}/d)$
3. If  $\mu \leq \mu_{lim}$  (Singly Reinforced):  
 $A_{st} = (0.5 \cdot f_{ck}/f_y) \cdot (1 - \sqrt{1 - 4.6 \cdot \mu / (f_{ck} \cdot b \cdot d^2)}) \cdot b \cdot d$
4. If  $\mu > \mu_{lim}$  (Doubly Reinforced):  
 $\mu_2 = \mu - \mu_{lim}; Asc = \mu_2 / (f_{sc} \cdot (d - d'))$   
 $A_{st} = A_{st1} + A_{st2}$
5. Enforce  $A_{st\_min} = 0.85 \cdot b \cdot d / f_y$

check\_pm\_interaction() (Bresler) Algorithm:

1.  $P_{uz} = 0.45 \cdot f_{ck} \cdot (A_g - A_{st}) + 0.75 \cdot f_y \cdot A_{st}$
2.  $\alpha = 1.0$  if  $P_u/P_{uz} \leq 0.2$ ,  $2.0$  if  $\geq 0.8$ , else interpolate
3. Compute  $M_{ux1}$ ,  $M_{uy1}$  using  $x_{u\_max}$  and lever arm ( $D - 2 \cdot d'$ )
4.  $Ratio = (M_{ux}/M_{ux1})^\alpha + (M_{uy}/M_{uy1})^\alpha$
5. PASS if ratio  $\leq 1.0$

**IS Code References:** IS 456:2000 CI 38.1, CI 39.3, CI 39.6, CI 40, Table 19; SP:16

## 8. REBAR\_DETAILER.PY

**File:** src/rebar\_detailer.py

Converts required steel areas into practical bar bending schedules. Handles column longitudinal bars (min 8 bars of 16mm), lateral ties, beam top/bottom bars with stirrup spacing, staircase waist slab design, and slab rebar mesh with weight calculations.

### Key Classes & Data Models

- **RebarResult:** main\_bars\_desc, links\_desc, steel\_area, weight\_kg\_per\_m, congestion\_status
- **BeamRebarResult:** top\_bars, bottom\_bars, stirrups, weight\_kg\_per\_m
- **StaircaseResult:** waist\_thk, main\_steel, dist\_steel, riser, tread
- **SlabRebarResult:** main\_steel, dist\_steel, thickness, weight\_kg\_per\_m2

### Key Functions

- detail\_column(b, d, pu, concrete, level, fy) -> RebarResult
- detail\_beam(b, d, span, load, moment, shear, fck) -> BeamRebarResult
- detail\_staircase(floor\_height, flight\_width, fy, fck) -> StaircaseResult
- detail\_slab(lx, ly, thk, load, fck) -> SlabRebarResult

### Core Algorithm (Step-by-Step)

detail\_column() Algorithm:

1.  $Asc_{req} = (Pu - 0.4 \cdot f_{ck} \cdot A_g) / (0.67 \cdot f_y - 0.4 \cdot f_{ck})$
2.  $Asc_{min} = 0.008 \cdot A_g$  (0.8% minimum)
3. Select bars: min 8 nos of 16mm, test [16,20,25,32mm]
4. Check congestion:  $provided\_area / A_g \leq 4.0\%$
5. Ties:  $pitch = \min(b, d, 16 \cdot main\_dia, 300mm)$
6. Compute unit weight (kg/m) including ties

detail\_beam() Algorithm:

1. Calculate factored moment  $M_u = 1.5 \cdot load \cdot span^2 / 8$
2. Design bottom bars (tension) using flexure formula
3. Top bars = max(50% of bottom, 2 bars of 12mm)
4. Stirrups:  $sv = 0.87 \cdot f_y \cdot A_{sv} \cdot d / V_{us}$
5.  $sv\_safe = \max(sv, 75mm)$ ;  $N = 1000 / sv\_safe$
6. Compute total weight per meter

tau\_c Lookup (IS 456 Table 19):

Grade-specific tables for M20 through M40

Linear interpolation between pt% entries

(Corrected: replaced  $\sqrt{f_{ck}/20}$  approximation)

**IS Code References:** IS 456:2000 Cl 26.5.1-3, Cl 33 (stairs), Table 19; SP 34 Section 7

## 9. FOUNDATION\_ENG.PY

**File:** src/foundation\_eng.py

Designs isolated square RC spread footings. Sizes plan area with 10% self-weight allowance, iteratively checks two-way punching shear at  $d/2$ , one-way shear at  $d$  from column face, and flexural bending at column face. Increases thickness in 50mm increments until all checks pass.

### Key Classes & Data Models

- `Footing(BaseModel):` `length_m`, `width_m`, `thickness_mm`, `area_m2`, `concrete_vol_m3`, `excavation_vol_m3`

### Key Functions

- `design_footing(axial_load, sbc, col_width, col_depth, concrete) -> Footing`
- `calculate_punching_shear_capacity(thickness, fck) -> float`

### Core Algorithm (Step-by-Step)

```
design_footing() Algorithm:
1. Design load = axial_load * 1.1 (10% self-weight)
2. Required area = design_load / SBC
3. Side = sqrt(area), round up to 50mm, min 1.5m
4. Initial thickness = 300mm (if load<=400kN) else 350mm
5. LOOP (increment thickness by 50mm):
  a. d = thickness - 50mm cover
  b. Punching shear at d/2: tau_v <= 0.25*sqrt(fck)
  c. One-way shear at d: tau_v <= 0.36 MPa
  d. Flexural depth check at column face
  e. If all pass: BREAK
6. Compute concrete vol and excavation vol (1.5m depth)
7. Return Footing with status
```

**IS Code References:** IS 456:2000 Cl 31.6.1-3 (shear), Cl 34.1.2 (bending moment)

## 10. SEISMIC.PY

**File:** src/seismic.py

IS 1893:2016 equivalent static seismic analysis. Calculates design horizontal coefficient  $A_h$ , spectral acceleration  $S_a/g$  for different soil types, seismic base shear  $V_b$ , vertical distribution of lateral forces, and storey shear/drift checks.

### Key Classes & Data Models

- `SeismicAnalyzer`: zone, importance\_factor, response\_reduction\_factor, soil\_type
- `SeismicResult`:  $A_h$ ,  $V_b$ , storey\_forces, storey\_shears, drifts

### Key Functions

- `get_spectral_acceleration(Ta)` -> float:  $S_a/g$  from IS 1893 Fig.2
- `calculate_base_shear(W, Ta)` -> float:  $V_b = A_h * W$
- `distribute_forces(storey_weights, storey_heights, Vb)` -> List

### Core Algorithm (Step-by-Step)

```
get_spectral_acceleration(Ta):
    If Ta < 0.10s:  $S_a/g = 1.0 + 15.0 * Ta$  (linear ramp)
    If 0.10 <= Ta <= 0.55s:  $S_a/g = 2.5$  (plateau)
    If 0.55 < Ta <= 4.0s:  $S_a/g = 1.36 / Ta$  (descending)
    (Corrected: was flat 2.5 for all Ta < 0.55)

calculate_base_shear():
    Ta = 0.075 * H^0.75 (RC frame)
    Ah = (Z/2) * (I/R) * (Sa/g)
    Vb = Ah * W

distribute_forces():
    Qi = Vb * (Wi * hi^2) / Sum(Wj * hj^2)
```

**IS Code References:** IS 1893:2016 (Part 1) Cl 6.4.2, Cl 7.6.1, Table 3, Fig. 2

## 11. WIND\_LOAD.PY

**File:** src/wind\_load.py

IS 875:2015 Part 3 wind load analysis. Calculates design wind speed  $V_z$  with terrain/height factors, design wind pressure  $p_z$ , external/internal pressure coefficients, and net wind force on building faces. Checks both  $+C_{pi}$  and  $-C_{pi}$  for worst-case envelope.

### Key Classes & Data Models

- WindLoadCalculator: basic\_wind\_speed, terrain\_category, building\_class
- WindResult: Vz\_profile, pz\_profile, wind\_forces\_per\_floor

### Key Functions

- calculate\_wind\_speed(height\_m) -> float:  $V_z = V_b * k_1 * k_2 * k_3 * k_4$
- calculate\_wind\_pressure(height\_m) -> float:  $p_z = 0.6 * V_z^2$
- calculate\_wind\_force(building\_dims, num\_floors) -> WindResult

### Core Algorithm (Step-by-Step)

```
calculate_wind_force() Algorithm:  
1. For each floor height z:  
    $V_z = V_b * k_1 * k_2(z) * k_3 * k_4$   
    $p_z = 0.6 * V_z^2$  (N/m2)  
2. External Cpe from Tables 4-7 (depends on L/W ratio)  
3. Internal Cpi (depends on building openings)  
4. Check BOTH  $+C_{pi}$  and  $-C_{pi}$ :  
    $P_{plus} = p_z * (C_{pe} - (+C_{pi}))$   
    $P_{minus} = p_z * (C_{pe} - (-C_{pi}))$   
    $P_{net} = \max(|P_{plus}|, |P_{minus}|)$   
5.  $F_{floor} = P_{net} * \text{tributary\_area\_at\_floor}$ 
```

**IS Code References:** IS 875 (Part 3):2015 Tables 4-7, Cl 6.2-6.3

## 12. LOAD\_COMBINATIONS.PY

**File:** src/load\_combinations.py

Generates IS 456:2000 Table 18 limit state load combinations. Includes strength ( $1.5DL+LL$ ), seismic ( $1.2/1.5$  factors), wind, and stability/uplift combinations ( $0.9DL \pm 1.5EQ/WL$ ). Calculates seismic weight with IS 1893 live load reduction factors.

### Key Classes & Data Models

- `LoadCombinationManager`: `include_wind`, `include_seismic`, `seismic_zone`
- `LoadCombination`: `name`, `combination_type`, `factors`, `is_critical`

### Key Functions

- `get_all_combinations()` -> `List[LoadCombination]`
- `get_governing_combination(DL, LL, WL, EQx, EQy)` -> `(combo, result)`
- `calculate_seismic_weight(DL, LL, intensity)` -> `SeismicWeightResult`
- `check_uplift_stability(DL, WL, EQ)` -> `(bool, FoS, message)`

### Core Algorithm (Step-by-Step)

```
Combinations generated:
Gravity:  $1.5 \cdot (DL + LL)$ 
Seismic:  $1.2 \cdot (DL + LL \pm EQ)$ ,  $1.5 \cdot (DL \pm EQ)$ 
Stability:  $0.9 \cdot DL \pm 1.5 \cdot EQ$  (both X and Y)
Wind: same pattern replacing EQ with WL
(Stability combos added in latest patch)

seismic_weight():
LL  $\leq 3$  kN/m2: include 25% LL
LL  $> 3$  kN/m2: include 50% LL
W = DL + factor * LL
```

**IS Code References:** IS 456:2000 Table 18; IS 1893:2016 Cl 6.3, Cl 7.4.3 Table 10

## 13. QUANTIFIER.PY

**File:** src/quantifier.py

Generates Bill of Materials (BOM), cost estimation, carbon footprint, and concrete mix breakdowns. Calculates concrete/steel quantities for columns, beams, slabs, footings, staircases, and masonry walls. Applies IS 1200 opening deduction rules and 4% steel wastage factor.

### Key Classes & Data Models

- `MaterialCost(BaseModel)`: `total_concrete_vol_m3`, `total_steel_weight_kg`, `total_cost_inr`, `cement_bags`, `sand_m3`, `aggregate_m3`
- `Quantifier`: `concrete_rate=5000`, `steel_rate=60`, `steel_wastage=4%`

### Key Functions

- `calculate_bom(columns, beams, footings, grid_mgr) -> MaterialCost`

### Core Algorithm (Step-by-Step)

```
calculate_bom() Algorithm:  
1. Column concrete:  $\text{Sum}(w * d * h)$  for all columns/floors  
2. Beam concrete:  $\text{Sum}(w * d * L)$  for all beams/floors  
3. Slab concrete:  $\text{Sum}(\text{panel\_area} * \text{thickness})$  (actual areas)  
(Corrected: uses slab_schedule areas, not bounding box)  
4. Footing concrete: Sum(footing.concrete_vol_m3)  
5. Steel: extract from rebar/beam/slab schedules  
Apply 4% wastage factor (rolling margin + cutting)  
6. Mix design (M25): cement=340 kg/m3, sand=0.39 m3/m3  
7. Masonry: deduct 33% for openings, ~500 bricks/m3  
8. Cost = concrete*5000 + steel*60 INR  
9. Carbon = concrete*300 + steel*1.85 kgCO2e  
10. Add 5% contingency to final cost
```

**IS Code References:** [IS 1200:1992 \(measurement\)](#), [IS 10262:2019 \(mix design\)](#), [CPWD/DSR 2023](#)

## 14. STABILITY.PY

**File:** src/stability.py

Structural stability and safety checks. Column and beam slenderness ratio verification, strong-column weak-beam checks per IS 13920, and overall building stability assessments. Includes division-by-zero guards for edge cases.

### Key Classes & Data Models

- `StabilityChecker`: performs various IS code stability checks

### Key Functions

- `check_column_slenderness(width, depth, height) -> dict`
- `check_beam_slenderness(width, depth, span) -> dict`

### Core Algorithm (Step-by-Step)

```
check_column_slenderness():  
1. Guard: if width<=0 or depth<=0 -> FAIL (zero dimension)  
2.  $\lambda = \text{effective\_length} / \min(\text{width}, \text{depth})$   
3. If  $\lambda \leq 12$ : Short column (OK)  
4. If  $\lambda > 12$ : Slender column (additional P-delta moment)  
5. Minimum  $\lambda \geq 5.0$  per IS 13920
```

IS Code References: IS 456:2000 Cl 25.1; IS 13920:2016 Cl 7.2.1 (strong column-weak beam)

---

## 15. MATERIALS.PY

**File:** src/materials.py

Defines structural material models as Pydantic BaseModel classes. Concrete (with fck, modulus, Poisson ratio, density) and Steel grades. Provides standard Indian construction material properties per IS 456.

### Key Classes & Data Models

- Concrete(BaseModel): name, density, modulus\_of\_elasticity, poissons\_ratio, fck, grade

### Key Functions

- Standard instantiation: Concrete(name="M25", fck=25.0, grade="M25", density=25000, modulus\_of\_elasticity=25000, poissons\_ratio=0.2)

### Core Algorithm (Step-by-Step)

```
Material property lookup:  
M20: fck=20, E=22360 MPa  
M25: fck=25, E=25000 MPa  
M30: fck=30, E=27386 MPa  
Fe415: fy=415, xu_max/d=0.48  
Fe500: fy=500, xu_max/d=0.46
```

IS Code References: IS 456:2000 Table 2 (concrete grades), IS 1786 (TMT rebar)

---

END OF CODE ARCHITECTURE REFERENCE

StructOptima v1.0.0 | 15 modules documented | September 2026